

NoSQL

- NoSQL is a non-relational Database, that does not require a fixed schema, avoids joins, and is easy to scale
- The concept of NoSQL databases became popular with Internet giants like Google, Facebook, Amazon, etc. who deal with huge volumes of data
- In the year 1998- Carlo Strozzi use the term NoSQL for his lightweight, open-source relational database
- NoSQL databases never follow the relational model it is either schema-free or has relaxed schemas

Features of NoSQL

1. Non-relational -

- NoSQL databases never follow the relational model
- Never provide tables with flat fixed-column records
- Work with self-contained aggregates or BLOBs
- Doesn't require object-relational mapping and data normalization

2. Schema-free -

- NoSQL databases are either schema-free or have relaxed schemas
- Do not require any sort of definition of the schema of the data

Features of NoSQL

3. Simple API -

- Offers easy to use interfaces for storage and querying data provided
- APIs allow low-level data manipulation & selection methods
- Text-based protocols mostly used with HTTP REST with JSON

4. Distributed -

- Multiple NoSQL databases can be executed in a distributed fashion
- Offers auto-scaling and fail-over capabilities
- Often ACID concept can be sacrificed for scalability and throughput
- Only providing eventual consistency
- Shared Nothing Architecture. This enables less coordination and higher distribution.

Categories of NoSQL

There are mainly four categories of NoSQL databases. Each of these categories has its unique attributes and limitations.

No specific database is better to solve all problems.

- Key-value Pair Based
- Column-oriented Graph
- Graphs based
- Document-oriented

Key Value Pair Based

- Data is stored in key/value pairs. It is designed in such a way to handle lots of data and heavy load.
- Key-value pair storage databases store data as a hash table where each key is unique, and the value can be a JSON, BLOB(Binary Large Objects), string, etc.
- It is one of the most basic types of NoSQL databases. This kind of NoSQL database is used as a collection, dictionaries, associative arrays, etc. Key value stores help the developer to store schema-less data.
- They work best for shopping cart contents.
- Redis, Dynamo, Riak are some examples of key-value store Databases. They are all based on Amazon's Dynamo paper.

Key Value Pair Based

Key	Value
Name	Joe Bloggs
Age	42
Occupation	Stunt Double
Height	175cm
Weight	77kg

Column-based

- Column-oriented databases work on columns and are based on BigTable paper by Google.
- Every column is treated separately. Values of single column databases are stored contiguously.
- They deliver high performance on aggregation queries like SUM, COUNT, AVG, MIN etc. as the data is readily available in a column.
- Column-based NoSQL databases are widely used to manage data warehouses, business intelligence, Library card catalogs,
- HBase, Cassandra, HBase, Hypertable are examples of column based database.

Column-based

ColumnFamily			
Row Key	Column Name		
	Key	Key	Key
	Value	Value	Value
	Column Name		
	Key	Key	Key
	Value	Value	Value

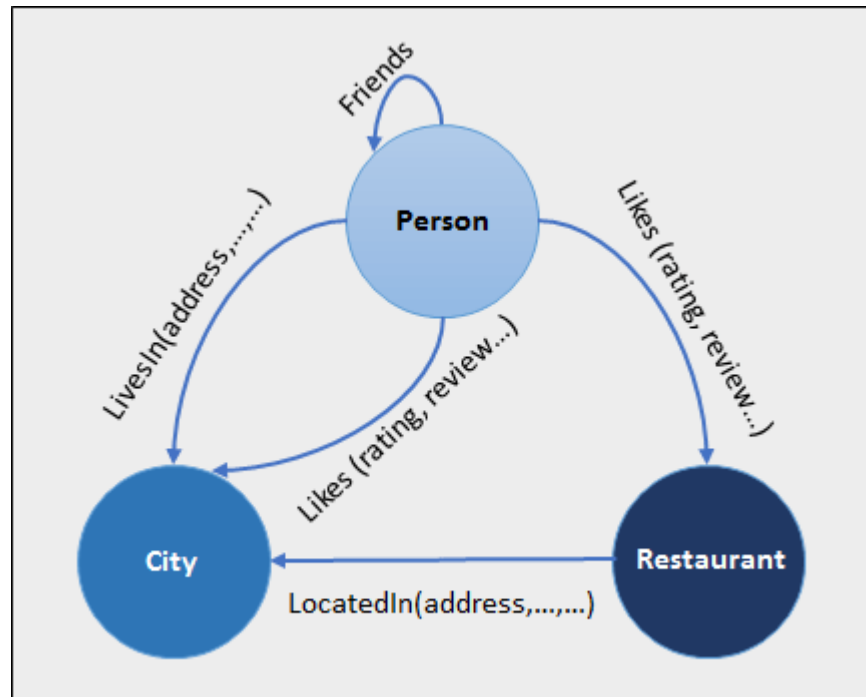
Document-Oriented

- Document-Oriented NoSQL DB stores and retrieves data as a key value pair but the value part is stored as a document.
- The document is stored in JSON or XML formats. The value is understood by the DB and can be queried.
- The document type is mostly used for CMS systems, blogging platforms, real-time analytics & e-commerce applications.
- It should not use for complex transactions which require multiple operations or queries against varying aggregate structures.

Graph-Based

- A graph type database stores entities as well the relations amongst those entities.
- The entity is stored as a node with the relationship as edges.
- An edge gives a relationship between nodes. Every node and edge has a unique identifier.
- Compared to a relational database where tables are loosely connected, a Graph database is a multi-relational in nature.
- Traversing relationship is fast as they are already captured into the DB, and there is no need to calculate them.
- Graph base database mostly used for social networks, logistics, spatial data.
- Neo4J, Infinite Graph, OrientDB, FlockDB are some popular graph-based databases.

Graph-Based



CAP Theorem

- CAP theorem is also called brewer's theorem. It states that is impossible for a distributed data store to offer more than two out of three guarantees

- Consistency
- Availability
- Partition Tolerance

- **Consistency:**

The data should remain consistent even after the execution of an operation. This means once data is written, any future read request should contain that data. For example, after updating the order status, all the clients should be able to see the same data.

CAP Theorem

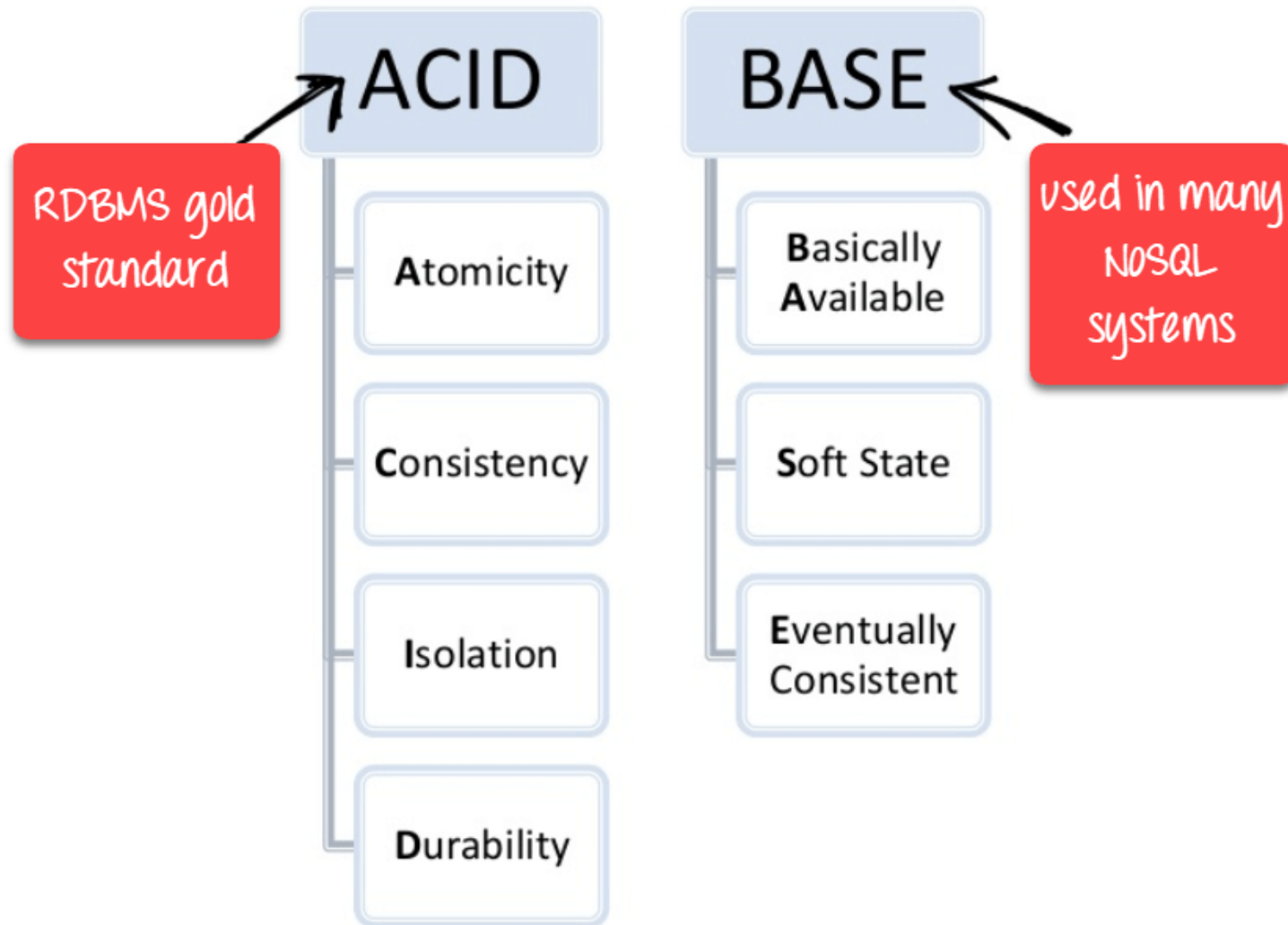
- **Availability:**

The database should always be available and responsive. It should not have any downtime.

- **Partition Tolerance:**

- Partition Tolerance means that the system should continue to function even if the communication among the servers is not stable.
- For example, the servers can be partitioned into multiple groups which may not communicate with each other.
- Here, if part of the database is unavailable, other parts are always unaffected.

BASE



BASE

- **BASE: Basically Available, Soft state, Eventual consistency**
- Basically, available means DB is available all the time as per CAP theorem
- Soft state means even without an input; the system state may change
- Eventual consistency means that the system will become consistent over time.

BASE

- Eventual Consistency - The term "**eventual consistency**" means to have copies of data on multiple machines to get high availability and scalability. Thus, changes made to any data item on one machine has to be propagated to other replicas.
- Data replication may not be instantaneous as some copies will be updated immediately while others in due course of time. These copies may be mutually, but in due course of time, they become consistent. Hence, the name eventual consistency.

Advantages of NoSQL

- Can be used as Primary or Analytic Data Source
- Big Data Capability
- No Single Point of Failure
- Easy Replication
- No Need for Separate Caching Layer
- It provides fast performance and horizontal scalability.
- Can handle structured, semi-structured, and unstructured data with equal effect
- Object-oriented programming which is easy to use and flexible
- NoSQL databases don't need a dedicated high-performance server

Create Database

The use Command

MongoDB `use DATABASE_NAME` is used to create database. The command will create a new database if it doesn't exist, otherwise it will return the existing database.

Syntax

```
use DATABASE_NAME
```

Drop Database

The dropDatabase() Method

MongoDB `db.dropDatabase()` command is used to drop a existing database.

Syntax

```
db.dropDatabase()
```

This will delete the selected database. If you have not selected any database, then it will delete default 'test' database.

Example

```
>use mydb
```

```
switched to db mydb
```

```
>db.dropDatabase()
```

```
>{ "dropped" : "mydb", "ok" : 1 }
```

The createCollection() Method

MongoDB `db.createCollection(name, options)` is used to create collection.

Syntax

```
db.createCollection(name, options)
```

Name String	Name of the collection to be created
-------------	--------------------------------------

Options Document	(Optional)	Specify options about memory size and indexing
------------------	------------	--

CreateCollection

Basic syntax of createCollection() method without options is as follows –

```
>use test
```

```
switched to db test
```

```
>db.createCollection("mycollection")
```

```
{ "ok" : 1 }
```

In MongoDB, you don't need to create collection. MongoDB creates collection automatically, when you insert some document.

```
>db.student.insert({"name" : "Arun"})
```

```
>show collections
```

```
mycollection
```

```
system.indexes
```

```
student
```

Insertion

To insert data into MongoDB collection, you need to use MongoDB's insert() or save() method.

Syntax

```
>db.COLLECTION_NAME.insert(document)
```

Example

```
db.post.insert([
  {
    title: 'MongoDB Overview',
    description: 'MongoDB is no sql database',
    tags: ['mongodb', 'database', 'NoSQL'],
    likes: 100
  }
])
```

Updation

The update() method updates the values in the existing document.

Syntax

```
db.COLLECTION_NAME.update(SELECTION_CRITERIA, UPDATED_DATA)
```

By default, MongoDB will update only a single document. To update multiple documents, you need to set a parameter 'multi' to true.

```
>db.mycol.update({'title':'MongoDB '},  
  {$set: {'title':'New MongoDB Overview '}}, {multi:true})
```

The find() Method

To query data from MongoDB collection, you need to use MongoDB's find() method.

Syntax

```
>db.COLLECTION_NAME.find()
```

find() method will display all the documents in a non-structured way.

The pretty() Method

To display the results in a formatted way, you can use pretty() method.

Syntax

```
>db.mycol.find().pretty()
```


AND in MongoDB

In the `find()` method, if you pass multiple keys by separating them by ',' then MongoDB treats it as AND condition.

Syntax

```
>db.mycol.find(  
  {  
    $and: [  
      {key1: value1}, {key2:value2}  
    ]  
  }  
)
```

OR in MongoDB

To query documents based on the OR condition, you need to use \$or keyword.

syntax of OR –

```
>db.mycol.find(  
  {  
    $or: [  
      {key1: value1}, {key2:value2}  
    ]  
  }  
)
```

The remove() Method

MongoDB's remove() method is used to remove a document from the collection.

Syntax

```
>db.COLLECTION_NAME.remove(DELETION_CRITTERIA)
```

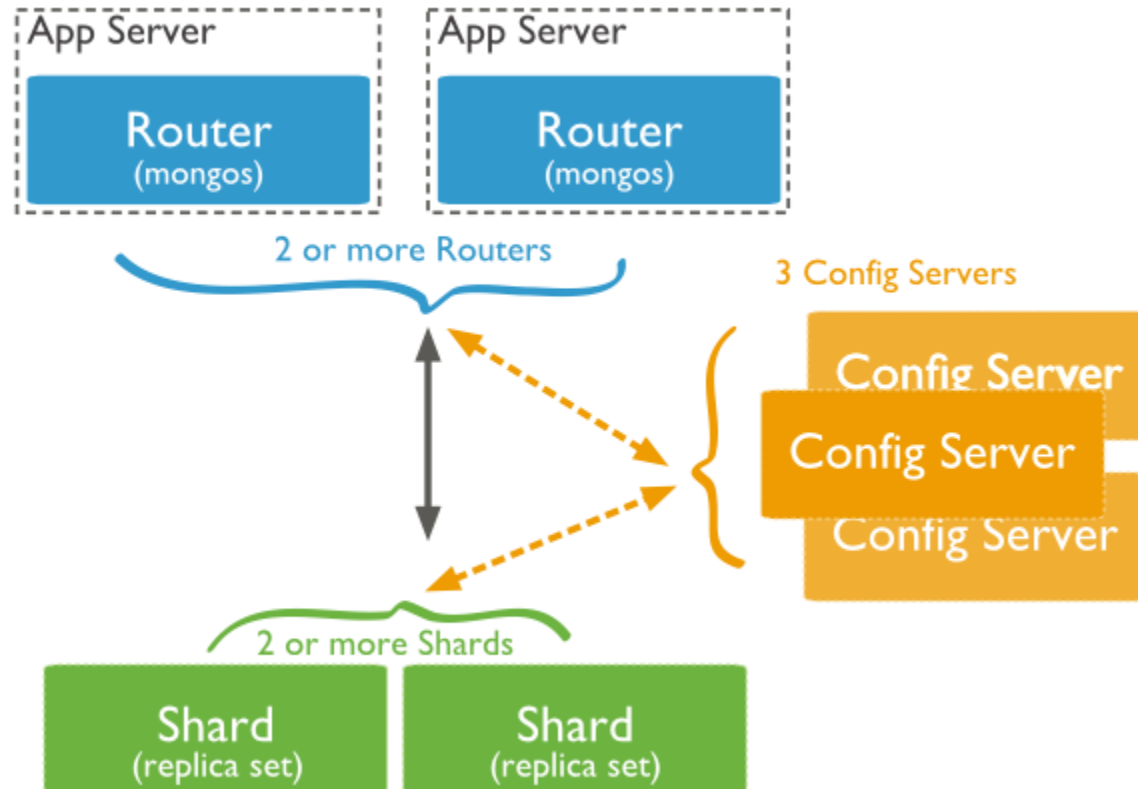
Example

```
>db.mycol.remove({'title':'MongoDB Overview'})
```

Sharding

- Sharding is the process of storing data records across multiple machines and it is MongoDB's approach to meeting the demands of data growth.
- As the size of the data increases, a single machine may not be sufficient to store the data nor provide an acceptable read and write throughput.
- Sharding solves the problem with horizontal scaling.

Sharding



Sharding

- **Shards** – Shards are used to store data. They provide high availability and data consistency. In production environment, each shard is a separate replica set.
- **Config Servers** – Config servers store the cluster's metadata. This data contains a mapping of the cluster's data set to the shards. The query router uses this metadata to target operations to specific shards.

Sharding

- **Query Routers** – Query routers are basically mongo instances, interface with client applications and direct operations to the appropriate shard. The query router processes and targets the operations to shards and then returns results to the clients.
- A sharded cluster can contain more than one query router to divide the client request load

Replication

- Replication is the process of synchronizing data across multiple servers.
- Replication provides redundancy and increases data availability with multiple copies of data on different database servers.
- Replication protects a database from the loss of a single server.

MongoDB Replication

- Replica set is a group of two or more nodes (generally minimum 3 nodes are required).
- In a replica set, one node is primary node and remaining nodes are secondary.
- All data replicates from primary to secondary node.
- At the time of automatic failover or maintenance, election establishes for primary and a new primary node is elected.
- After the recovery of failed node, it again join the replica set and works as a secondary node

Sharding

